

Lecture - 5

(Operator overloading, Numpy)

- $x = (1, 2)^T$, $y = (10, 20)^T$, $z = 2x + y$
 Problem $\rightarrow$ python does not treat list as mathematical vectors.

$\downarrow$

```
class TBVEC:
    def __init__(self, x1, x2):
        self.x1 = x1
        self.x2 = x2
    def add(self, other):
        return TBVEC(self.x1 + other.x1,
                      self.x2 + other.x2)
    def scalar_multiply(self, oth):
        return TBVEC(self.x1 * oth,
                      self.x2 * oth)
```

- Numpy provides efficient numerical array obj. & operations.

$x = \text{np.array}([1, 2, 3])$

$\rightarrow$ numpy.ndarray

$y = \text{np.array}([1, 2], [3, 4])$

$y.\text{shape} \Rightarrow (2, 2)$

$\rightarrow$ rows, columns

$y.\text{size} \Rightarrow (6)$

$\rightarrow$ no. of elements

$\text{np.zeros}(2, 3) \Rightarrow \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$

$\text{np.ones}(1, 2) \Rightarrow [1, 1]$

$\text{np.full}((2, 2), 7) \Rightarrow \begin{bmatrix} 7 & 7 \\ 7 & 7 \end{bmatrix}$

$\text{np.eye}(2) \Rightarrow \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$

$\text{np.linspace}(0, 9, 10)$

$\rightarrow [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]$

`np.save('array.npy', f)`
save the array `f` to a `array.npy` file

`np.load('array.npy')`
`h = g.astype('int')`

↪ changing datatype to integer

@ → matrix multiplication

`np.dot(A, B)` → dot product

`np.arange(16).reshape(4, 4)`

↪ `[:, 0]`, extracts first col.

- Linear Algebra using numpy

`A = np.array([2, 1], [1, 3])`

`b = np.array([5, 6])`

`x = np.linalg.solve(A, b)`

you can check that $A @ x = b$

for transpose just use `A.T`

`detA = np.linalg.det(A)`

`Ainv = np.linalg.inv(A)`

`norm-x = np.linalg.norm(x)`

eigenvalues,

`eigenvectors = np.linalg.eig(A)`